

This is popularly referred to as the *adjoint* of $\mathcal{G}_h$, although it seems something of an abuse of mathematical language to refer to it in this way. For the flow map $\mathcal{F}_h$, we know that the inverse map is precisely $\mathcal{F}_{-h}$, so $\mathcal{F}_h^\dagger = \mathcal{F}_h$, i.e. the flow map is in the normal sense “self-adjoint,” i.e. *symmetric*. However, such a property does not hold in the general case. In particular, consider Euler’s method

$$\mathbf{Z} = \mathbf{z} + hf(\mathbf{z}).$$

The adjoint method is defined by

$$\mathbf{Z} = \mathbf{z} + hf(\mathbf{Z}),$$

and where the first was explicit, the second is implicit (it is the so-called *backward Euler* method).

The method (2.18)–(2.19) is called the Symplectic Euler method. Its adjoint method has a similar structure:

$$\mathbf{Q} = \mathbf{q} + h\mathbf{M}^{-1}\mathbf{p}, \quad (2.22)$$

$$\mathbf{P} = \mathbf{p} + h\mathbf{F}(\mathbf{Q}). \quad (2.23)$$

Comparing with (2.18)–(2.19), we see that (2.22)–(2.23) is also explicit.

Given a method $\mathcal{G}_h$ with adjoint method $\mathcal{G}_h^\dagger$, it is possible to obtain the adjoint of the adjoint method $\mathcal{G}_h^{\dagger\dagger}$, but, as we might expect, the adjoint of the adjoint is the original method:

$$\mathcal{G}_h^{\dagger\dagger} = [\mathcal{G}_h^\dagger]^{-1} = [\mathcal{G}_h^{-1}]^{-1} = \mathcal{G}_h.$$

2.4 Building Symplectic Integrators

Symplectic integrators may be constructed in several ways. First, we may look within standard classes of methods such as the family of Runge-Kutta schemes to see if there are choices of coefficients which make the methods automatically conserve the symplectic 2-form. A second, more direct approach is based on *splitting*. The idea of splitting methods, often referred to in the literature as *Lie-Trotter methods*, is that we divide the Hamiltonian into parts, and determine the flow maps (or, in some cases, approximate flow maps) for the parts, then compose the maps to define numerical methods for the whole system.